

STUDYWAVE · TEAM IMPLEMENTATION SPEC

구현 가이드

파일명을 기준으로 한 데이터 흐름 · 통합 방법 · 역할별 작업 순서

PDF / 코드 입력 → AgentState → 북습노트 + 퀴즈 + Streamlit / Obsidian .md

구분	담당	핵심 폴더	Graph Node
A	감독 · 라우팅 · 검증	graph/ , schemas/state.py	route, verify
B	파싱 · 전처리 · 적재	workers/parser/ , store/	parse, retrieve
C	개념 · 코드 · 교차매핑	workers/analysis/	analyze
D	퀴즈 · 마크다운	workers/formatter/	format
UI	업로드 · 탭 · 내보내기	app.py, ui/	run_pipeline 호출

1. 이 문서가 다루는 것

이 문서는 StudyWave 저장소의 파일 이름을 기준으로, 데이터가 어디서 만들어지고 어느 파일로 넘어가며, 마지막에 어떻게 한 화면·한 문서로 합쳐지는지를 고정한다. 구현 본문은 각 담당자가 자신의 .py 에 채운다. 필드 이름과 반환 key 를 바꾸면 다른 역할의 코드가 깨지므로, 스키마 변경은 전원 합의 후에만 한다.

2. 최종 구현이 의미하는 것

사용자가 Streamlit 에서 PDF와 코드 파일 중 하나 또는 둘을 올리면, LangGraph 가 입력 조합을 보고 Worker 를 순서대로 실행한다. 결과는 탭 세 개(개념 요약, 코드 흐름 분석, 복습 퀴즈)로 보이고, 같은 내용을 Obsidian 호환 .md 로 내려받는다. 퀴즈는 3~5문항이며 정답·해설은 토글로 숨긴다.

- 입력: PDF만 / 코드만 / 둘 다. 파일 분류는 graph/router.py 의 detect_route.
- 전달: 전 구간이 schemas/state.py 의 AgentState 딕셔너리.
- 저장: 파싱된 Chunk 는 store/vector_store.py (PostgreSQL + pgvector).
- 생성: C가 분석 항목을 만들고 D가 퀴즈와 마크다운을 만든다.
- 검증: A가 출처·스키마·route 충족 여부를 확인한 뒤 UI에 넘긴다.

3. 한 장의 호출 흐름 (파일명)

아래는 런타임에 실제로 import · 호출되는 순서다. 화살표의 왼쪽 파일이 오른쪽 파일을 부른다.

```
app.py
  ■■ ui/app.py
    ■■ ■■■■ ■■■■ FileInfo ■■■■ ■■
    ■■ graph/supervisor.py → run_pipeline(files)

graph/supervisor.py (StateGraph.compile → invoke)
  1) graph/router.py route_input_node
  2) workers/parser/nodes.py parse_files_node
    ■■ workers/parser/pdf_parser.py
    ■■ workers/parser/code_parser.py
    ■■ workers/parser/preprocess.py
    ■■ store/vector_store.py upsert_chunks
  3) workers/parser/nodes.py retrieve_chunks_node
    ■■ store/vector_store.py retrieve_vector_documents
  4) workers/analysis/nodes.py analyze_node
    ■■ workers/analysis/prompts.py
    ■■ workers/analysis/concept.py
    ■■ workers/analysis/code_flow.py
    ■■ workers/analysis/cross_reference.py
    ■■ workers/analysis/troubleshooting.py
  5) workers/formatter/nodes.py format_node
    ■■ workers/formatter/quiz.py
    ■■ workers/formatter/markdown.py
  6) graph/verify.py verify_node

ui/tabs.py ← ■■■ AgentState
  ■ [■■ ■■] [■■ ■■ ■■] [■■ ■■]
  + final_markdown ■■■■
```

그림 1. 엔트리 파일부터 Worker, 저장소, 검증, UI까지 파일명으로 이은 실행 경로.

4. 통합이 되는 원리

4.1 AgentState 가 유일한 버스

graph/supervisor.py 는 Worker 끼리 직접 함수를 호출하지 않는다. 각 Node 는 state 를 받아 자신이 만든 key 만 dict 로 반환하고, LangGraph 가 그 key 를 AgentState 에 합친다. 그래서 B가 끝나기 전에도 C는

mocks/data.py 의 retrieved_chunks 로 프롬프트를 짤 수 있다.

Node	파일	반드시 반환하는 key
route	graph/router.py	route, status
parse	workers/parser/nodes.py	files, parsed_chunks, status
retrieve	workers/parser/nodes.py	retrieved_chunks, status
analyze	workers/analysis/nodes.py	concept_summary, code_analysis, cross_references, troubleshooting, status
format	workers/formatter/nodes.py	quiz_items, final_markdown, status
verify	graph/verify.py	verification, quiz_items, status, errors

4.2 Mock 으로 먼저 파이프를 잠근다

graph/supervisor.py 의 USE MOCK = True 이면 B/C/D 함수 본문이 비어 있어도 python run_graph.py 가 키 이름 흐름을 보여 준다. 자신의 Node 를 실제 함수로 바꾼 뒤 USE MOCK = False 로 전환하면 supervisor 가 실제 모듈을 import 한다. 주간 통합은 이 스위치 하나로 한다.

4.3 공유 계약 파일

파일	누가 쓰나	역할
schemas/state.py	A가 소유, 전원 import	AgentState TypedDict. 필드 추가·삭제는 회의 안건.
schemas/files.py	B가 채움, C·D·A가 읽음	FileInfo, Chunk, RetrievedChunk. chunk_id 규칙 고정.
schemas/analysis.py	C가 채움, D·UI가 읽음	ConceptItem, CodeAnalysisItem, CrossReferenceltem, TroubleshootingItem.
schemas/quiz.py	D가 채움, A·UI가 읽음	QuizItem. answer 는 보기 문자열 그대로.
schemas/errors.py	A가 채움, UI가 읽음	NodeError, VerificationReport.
config.py	전원	GEMINI_API_KEY, LLM_PROVIDER, VECTOR_BACKEND, TOP_K.
store/vector_store.py	B 쓰기, retrieve가 읽기	initialize / upsert / retrieve. 본문은 스터디에서 구현.
mocks/data.py	전원 참조	입출력 예시 JSON 역할. 병렬 개발의 기준 샘플.

5. 공통 규칙 (전원)

- 1 리스트가 비면 None 이나 key 생략 대신 [] 를 넣는다. 문자열 미생성만 None.
- 2 출처 번호는 1부터. PDF는 page_number, 코드는 start_line / end_line.
- 3 chunk_id = {document_id}:chunk: {index:04d}, document_id = {request_id}: {파일명}.
- 4 분석·퀴즈 항목에는 source_chunk_ids 를 넣는다. 근거 없는 문장은 쓰지 않는다.
- 5 API 키는 .env 의 GEMINI_API_KEY 만 사용한다. 코드와 채팅과 git 에 키를 붙이지 않는다.
- 6 임베딩 모델이나 차원을 바꾸면 벡터 테이블을 처음부터 다시 만든다.
- 7 설치: 프로젝트 루트에서 pip install -r requirements.txt.

6. 환경 파일

```
# ■■■■ ■■ .env
```

```
GEMINI_API_KEY=■■■■■■■■■■■
GEMINI_MODEL=gemini-3.7-flash
GEMINI_EMBEDDING_MODEL=gemini-embedding-2
LLM_PROVIDER=gemini
EMBEDDING_PROVIDER=gemini
VECTOR_BACKEND=pgvector
DATABASE_URL=postgresql://localhost:5432/studywave
```

```
# █ █ OSS █ █ █ █
# LLM_PROVIDER=ollama
# OLLAMA_BASE_URL=http://localhost:11434
# OLLAMA_CHAT_MODEL=gemma3:4b
```

모델 객체는 `config.get_chat_model()` 과 `config.get_embeddings()` 에서만 만든다. Worker 파일마다 Client 를 새로 열지 않는다.

7. A 작업 가이드 — 감독 / 라우팅 / 검증

A는 네 Worker 를 한 그래프에 꽂고, 입력 종류에 따라 길을 나누며, 나온 결과가 스키마와 출처 규칙을 지키는지 마지막에 확인한다. LLM으로 노트를 쓰는 역할이 아니다. 품질의 문은 규칙을 통과했는가다.

7.1 담당 파일

파일	지금 상태	A가 할 일
graph/supervisor.py	직선 Edge + Mock Node 있음	USE MOCK=False 경로 유지. retry 시 add_conditional_edges 추가.
graph/router.py	detect_route 구현됨	확장자-source_type 규칙 유지. LLM 라우팅은 선택 과제.
graph/verify.py	함수 시그니처만	citation / quiz / route 검사와 verify_node 본문.
schemas/state.py	필드 초안 있음	필드 타입 확정. 바꾸면 슬랙/노선에 공지.
schemas/errors.py	모델 있음	error_code 목록을 짧게 고정 (PARSE_FAIL, EMPTY_SECTION 등).
run_graph.py	엔트리	통합 전에 항상 여기서 키 흐름을 출력해 확인.

7.2 구현 순서

- 1 python run_graph.py 를 실행해 route / status / chunks / quiz / passed 가 출력되는지 확인한다. (langgraph 설치 필요)
- 2 graph/router.py 의 detect_route 를 PDF만, 코드만, 둘 다 세 케이스로 손으로 호출해 본다.
- 3 graph/verify.py 의 collect_known_chunk_ids 부터 채운다. parsed_chunks 와 retrieved_chunks 의 chunk_id 집합이면 충분하다.
- 4 validate_citations: C·D 항목의 source_chunk_ids 가 그 집합 안에 있는지 본다. 없는 id 는 issues 문자열로 남긴다.
- 5 validate_quiz_items: quiz_type 이 mcq 이면 answer 가 options 에 있어야 한다. 실패한 문항은 dropped_quiz_ids 에 인덱스를 넣는다.
- 6 validate_route_outputs: pdf_only 인데 concept_summary 가 {} 이면 실패, code_only 인데 code_analysis 가 {} 이면 실패, both 인데 cross_references 가 {} 이면 실패.
- 7 verify_node 에서 위 세 함수를 모아 VerificationReport 를 만들고 status 를 completed 또는 failed 로 둔다.
- 8 config.NODE_FAILURE_POLICY 가 retry 이고 retryable 에러가 있으면 supervisor 에 add_conditional_edges 를 추가한다. 기본값은 partial 이라 부분 결과를 UI에 넘겨도 된다.

7.3 완료 기준

- mocks/data.py 를 넣은 invoke 결과가 예외 없이 끝난다.
- 고의로 잘못된 chunk_id 를 넣으면 verification.passed 가 False 가 된다.
- B/C/D 가 NotImplementedError 인 동안에도 USE MOCK=True 경로가 살아 있다.
- USE MOCK=False 일 때 import 경로가 workers/*.nodes 와 graph.verify 를 가리킨다.

7.4 강의에서 그대로 가져올 API

day2 agentEx3/Ex4, day3 agentEx5 의 StateGraph, START, END, add_node, add_edge, compile, invoke. MessagesState 와 ToolNode(직원 조회)는 이 서비스 그래프에 넣지 않는다. 검증에 LLM을 쓸 때만 config.get_chat_model(role="verify") 를 호출한다. 규칙은 LLM보다 먼저 돌린다.

8. B 작업 가이드 — 파싱 / 전처리 / 검색

B는 원본 파일을 AgentState 가 이해할 수 있는 Chunk 리스트로 바꾸고, 벡터 저장소에 넣은 뒤, 같은 request_id 범위에서만 다시 꺼내 C에게 넘긴다. 요약은 쓰지 않는다. 깨끗하고 출처가 붙은 조각만 만든다.

8.1 담당 파일

파일	입력	출력
workers/parser/pdf_parser.py	FileInfo (source_type=pdf)	페이지 또는 문단 단위 Chunk, page_number 채움
workers/parser/code_parser.py	FileInfo (source_type=code)	함수/클래스/슬라이딩 윈도우 Chunk, start_line-end_line
workers/parser/preprocess.py	원본 FileInfo / Chunk	title-size_bytes 등 메타, 공백 정리된 content
workers/parser/nodes.py	AgentState.files	parse: files+parsed_chunks / retrieve: retrieved_chunks
store/vector_store.py	request_id + Chunk 리스트	upsert 성공, retrieve 시 RetrievedChunk (distance, vector_rank)

8.2 구현 순서

- 1 schemas/files.py 를 읽고 FileInfo, Chunk, RetrievedChunk 필드를 외운다. 새 필드를 만들기 전에 기존 필드로 되는지 본다.
- 2 make_document_id / make_chunk_id 는 nodes.py 에 이미 있다. 파서에서 id 를 새로 발명하지 말고 이 함수를 쓴다.
- 3 pdf_parser.parse_pdf: pypdf 또는 pdfplumber 로 페이지 텍스트를 뽑는다. 한 페이지를 한 Chunk 로 시작해도 된다. page_number 는 1부터.
- 4 목차(TOC)가 있으면 별도 Chunk 로 넣거나 metadata title 에 반영한다. 표가 깨지면 텍스트만이라도 살린다.
- 5 code_parser.parse_code: ast.parse 로 FunctionDef / ClassDef 의 lineno, end_lineno 를 쓴다. 파싱이 실패하는 파일은 일정 줄 수 슬라이딩 윈도우로 풀백한다.
- 6 preprocess.extract_metadata 로 파일 크기, 코드 줄 수, PDF title 을 FileInfo 에 채운다. refine_text 는 연속 공백과 제어 문자만 제거한다. 한글과 코드 기호는 지우지 않는다.
- 7 parse_files_node: files 를 순회하며 pdf/code 를 고르고, document_id 를 채운 뒤 parsed_chunks 를 반환한다. 그 다음 store.upsert_chunks(request_id, chunks) 를 호출한다.
- 8 store/vector_store.py: 합의는 pgvector. initialize 에서 확장과 테이블을 만든다. 컬럼에 request_id, document_id, source_type, page_number, start_line, end_line, content, embedding 을 둔다.
- 9 retrieve_chunks_node: 쿼리 문장은 파일명과 앞쪽 Chunk 를 이어 만들어도 된다. 반드시 request_id 로 필터한다. 결과는 RetrievedChunk 리스트.
- 10 config.get_embeddings() 로 벡터를 만든다. 기본은 gemini-embedding-2, 차원 768. nomic-embed-text 도 768. bge-m3 는 1024 이므로 EMBEDDING_DIMENSION 과 테이블을 함께 바꾼다.

8.3 완료 기준

- 샘플 PDF 한 장과 .py 한 파일을 넣었을 때 parsed_chunks 가 1개 이상이다.
- 모든 Chunk 에 chunk_id, document_id, source_type, content 가 있다. PDF에는 page_number, 코드에는 줄 번호가 있다.
- 다른 request_id 로 검색해도 이전 자료가 나오지 않는다.
- C는 nodes 를 몰라도 mocks 의 RetrievedChunk 형태와 실제 retrieve 결과가 같다.

8.4 강의에서 그대로 가져올 API

day1 retrieve_vector_documents 의 collection.query(include=documents, metadatas, distances). day3 initialize_vector_db, similarity_search_with_score, metadata filter. 강의는 Chroma PersistentClient 다. 함수

이름만 맞추면 VECTOR_BACKEND=chroma 로 연습한 뒤 pgvector 로 옮길 수 있다. 직원 조회 속제 SQL 은 복사하지 않는다.

9. C 작업 가이드 — 개념 / 코드 흐름 / 교차 매핑

C는 retrieved_chunks 만 보고 분석 항목을 만든다. 원본 PDF를 다시 열지 않는다. 퀴즈와 최종 .md 는 D의 일이다. C가 채우는 key 는 concept_summary, code_analysis, cross_references, troubleshooting 네 개다.

9.1 담당 파일

파일	하는 일	Pydantic 결과
workers/analysis/prompts.py	LangChain PromptTemplate 문자열	각 produce_* 가 사용
workers/analysis/concept.py	교안 핵심 개념 요약	list(ConceptItem)
workers/analysis/code_flow.py	실행 순서, 데이터 변환, 문법	list(CodeAnalysisItem)
workers/analysis/cross_reference.py	이 코드는 교안 p.X 의 무엇이다	list(CrossReferenceItem)
workers/analysis/troubleshooting.py	자주 나는 실수와 고치는 법	list(TroubleshootingItem)
workers/analysis/nodes.py	route 를 보고 위 함수를 조합	analyze_node 반환 dict
schemas/analysis.py	AnalysisLLMOutput 묶음 스키마	structured output 대상

9.2 구현 순서

- 1 mocks/data.py 의 MOCK_RETRIEVED_CHUNKS 를 입력으로 삼아, LLM 없이 ConceptItem 을 손으로 하나 만들어 model_validate 가 통과하는지 본다. 스키마를 먼저 몸에 익힌다.
- 2 build_evidence_context: 각 Chunk 를 (chunk_id | pdf p.N 또는 code Lx-Ly) 헤더 + content 블록으로 이어 붙인다. 토큰이 커지면 상위 TOP_K 만 쓴다.
- 3 prompts.py 에 네 개의 지시문을 적는다. 공통 문장: 근거 블록에 없는 사실은 쓰지 말 것, 각 항목에 source_chunk_ids 를 넣을 것, JSON만 출력할 것.
- 4 config.get_chat_model(role="generate") 로 Gemini 를 받는다. 강의의 ChatGoogleGenerativeAI 또는 interactions.create + response_format schema=AnalysisLLMOutput.model_json_schema() 중 팀에서 하나를 고른다.
- 5 concept.py: PDF Chunk 가 있을 때만 호출. title + summary + source_chunk_ids.
- 6 code_flow.py: 코드 Chunk 가 있을 때만 호출. 실행 순서와 핵심 문법(예: 필터 연산자, where)을 설명한다.
- 7 cross_reference.py: route 가 both 일 때만. theory_chunk_ids 와 code_chunk_ids 를 함께 넣는다. 문장 예시는 “이 코드의 WHERE 는 교안 p.3 메타데이터 필터를 구현한다.”
- 8 troubleshooting.py: 초기에 흔한 실패(필터 누락, 차원 불일치)를 박스 형태로. source_chunk_ids 는 있으면 넣고 없으면 {}.
- 9 analyze_node: route 가 pdf_only 면 개념만, code_only 면 코드만, both 면 네 리스트 모두. 쓰지 않는 키는 {} 로 반환한다.

9.3 route 와 산출 대응

route	concept_summary	code_analysis	cross_references	troubleshooting
pdf_only	1개 이상	{}	{}	{ } 또는 선택
code_only	{}	1개 이상	{}	{ } 또는 선택
both	1개 이상	1개 이상	1개 이상	있으면 채움

9.4 완료 기준

- MOCK_RETRIEVED_CHUNKS 만으로 AnalysisLLMOutput 이 파싱된다.
- 모든 ConceptItem / CodeAnalysisItem 에 source_chunk_ids 가 1개 이상이다.
- both 샘플에서 cross_references 가 PDF chunk 와 code chunk 를 동시에 가리킨다.
- D는 C 함수를 import 하지 않고 AgentState 의 네 리스트만 읽어도 마크다운을 만들 수 있다.

9.5 강의에서 그대로 가져올 API

day1 GraphQueryPlan.model_json_schema 와 model_validate_json. day1 build_fused_evidence_context / generate_answer 의 “근거 없는 사실은 쓰지 마라”. day2 PromptTemplate, ChatGoogleGenerativeAI. Hybrid RAG fusion 속제 로직은 복사하지 않는다. 검색은 B가 끝낸 retrieved_chunks 를 쓰는 것으로 대체한다.

10. D 작업 가이드 — 퀴즈 / 마크다운

D는 C가 남긴 분석 리스트와 `retrieved_chunks` 를 읽어 복습 문항 3~5개와 Obsidian/Notion 호환 마크다운 한 편을 만든다. 파싱과 개념 추출을 다시 하지 않는다. UI가 그릴 수 있도록 `quiz_items` 와 `final_markdown` 두 key 만 책임진다.

10.1 담당 파일

파일	하는 일	완료물
<code>workers/formatter/quiz.py</code>	주관·객관 혼합 3~5문항 + 해설	<code>list(QuizItem)</code>
<code>workers/formatter/markdown.py</code>	YAML + 섹션 + 토글 + 콜아웃	<code>final_markdown</code> 문자열
<code>workers/formatter/nodes.py</code>	두 함수를 모아 Graph Node	<code>quiz_items</code> , <code>final_markdown</code> , <code>status</code>
<code>schemas/quiz.py</code>	문항 스키마	<code>QuizItem</code>

10.2 구현 순서

- 1 `schemas/quiz.py` 를 연다. `quiz_type` 은 `mcq` / `short` / `true_false`. `mcq` 는 `options` 4개를 목표로 한다. `answer` 는 “1번”이 아니라 `options` 안의 문자열과 완전히 같다.
- 2 `mocks` 의 `MOCK_CONCEPT_SUMMARY`, `MOCK_CODE_ANALYSIS` 만으로 `QuizItem` 두 개를 손으로 만들어 `model_validate` 한다.
- 3 `produce_quiz_items`: C 산출과 `retrieved_chunks` 를 프롬프트에 넣고 Gemini 로 3~5문항을 받는다. 각 문항에 `source_chunk_ids` 를 넣는다. 객관과 주관 섞는다.
- 4 `format_markdown` 목차 순서를 고정한다. YAML → 개념 → 코드 흐름 → 이론-코드 연결 → Troubleshooting → Quiz. 빈 섹션은 `config.EMPTY_SECTION_POLICY` 가 `omit` 이면 제목째 뺀다.
- 5 YAML 예: `request_id`, `route`, `tags`: `[studywave, 주제]`. Obsidian 이 읽는 `frontmatter` 형식을 지킨다.
- 6 퀴즈 블록은 HTML details 또는 마크다운 토글로 정답·해설을 접는다. 콜아웃은 `> (!NOTE)`, `> (!WARNING)` 형식을 쓴다. Streamlit 탭도 같은 데이터를 쓰므로 문장 내용을 두 벌로 만들지 않는다.
- 7 `format_node` 는 `quiz` 와 `markdown` 만 호출해 `dict` 로 반환한다. `concept_summary` 를 여기서 비우지 않는다.

10.3 마크다운 뼈대

```

---
request_id: req-demo-001
route: both
tags: [studywave, langgraph]
---

# ■■ ■■
...

# ■■ ■■ ■■
...

# ■■-■■ ■■
...

# Troubleshooting

> [!WARNING]
> ■■ ■■■ ■■■ ■■ ■■ ■■■ ■■■.

# ■■ ■■

<details>

```

```
<summary>Q1. LangGraph ■■ ■■ ■■■ ■■■■ ■■■■?</summary>
```

```
■■: StateGraph
```

```
■■: StateGraph ■ Node/Edge ■ ■■■ ■■■ compile ■■.  
</details>
```

10.4 완료 기준

- 문항 수가 3 이상 5 이하이다.
- mcq 의 answer 가 options 중 하나와 같다.
- final_markdown 이 --- 로 시작하고 # 복습 퀴즈 섹션을 포함한다.
- ui/tabs.py 는 quiz_items 만으로 토글을 그릴 수 있고, 다운로드 버튼은 final_markdown 만 저장한다.

10.5 강의에서 그대로 가져올 API

C와 같은 Gemini structured output. Pydantic QuizItem 리스트를 schema 로 넘긴다. 검증(출처 존재 여부)은 A의 graph/verify.py 가 담당하므로 D는 생성과 포맷에 집중한다.

11. UI 파일 — 1~2일 골격

프론트는 Streamlit. 그래프가 Mock 이어도 업로드 칸과 탭과 다운로드 버튼이 보이면 된다.

파일	역할	연동
app.py	streamlit run app.py 진입점	ui/app.py 의 main()
ui/app.py	파일 업로더, 생성 버튼, 예러 표시	graph.supervisor.run_pipeline
ui/tabs.py	세 탭 렌더	concept_summary / code_analysis / quiz_items

업로더는 PDF와 .py 를 받는다. 선택한 파일로 FileInfo 를 만들 때 source_type 만 맞으면 router 가 route 를 정한다. 생성 버튼은 run_pipeline 한 번이다. 결과가 오면 st.tabs 로 세 화면을 그리고 st.download_button 으로 .md 를 저장한다.

12. 주간 통합 순서

- 1 A가 run_graph.py 로 Mock 파이프가 살아 있는지 확인한다.
- 2 B가 parse_files_node 만 실제 함수로 바꾸고, supervisor 에서 parse 노드만 교체해 본다.
- 3 C가 mocks 의 retrieved_chunks 로 analyze_node 를 단위 실행한다. 그래프에 붙이기 전에 함수 단위로 확인한다.
- 4 D가 mocks 의 분석 결과로 format_node 를 단위 실행한다.
- 5 A가 verify_node 를 실제 함수로 교체하고, 잘못된 인용을 넣은 샘플로 실패를 확인한다.
- 6 USE MOCK=False 로 전환한 뒤 PDF+코드 한 세트를 올린다. 세 탭과 .md 가 같은 내용을 가리키면 통합 완료다.

13. 파일 점검표

파일	담당	통합 시 확인
graph/supervisor.py	A	6개 Node 가 이 파일에서만 연결되는가
graph/router.py	A	세 가지 입력 조합이 나오는가
graph/verify.py	A	잘못된 인용을 걸러 내는가
workers/parser/pdf_parser.py	B	page_number 가 비어 있지 않은가
workers/parser/code_parser.py	B	줄 번호가 비어 있지 않은가
workers/parser/preprocess.py	B	한글·코드 기호가 살아 있는가
workers/parser/nodes.py	B	반환 key 가 문서와 같은가
store/vector_store.py	B·공유	request_id 필터가 있는가
workers/analysis/prompts.py	C	근거 없는 사실 금지 문장이 있는가
workers/analysis/concept.py	C	source_chunk_ids 가 있는가
workers/analysis/code_flow.py	C	실행 순서가 문장에 드러나는가
workers/analysis/cross_reference.py	C	PDF id 와 코드 id 가 같이 있는가
workers/analysis/troubleshooting.py	C	증상-원인-조치 세 칸인가
workers/analysis/nodes.py	C	route 별 빈 리스트 규칙이 맞는가
workers/formatter/quiz.py	D	3~5문항, answer=보기 문자열인가

파일	담당	통합 시 확인
workers/formatter/markdown.py	D	YAML 과 토글이 있는가
workers/formatter/nodes.py	D	C의 리스트를 지우지 않는가
ui/app.py / ui/tabs.py	UI	세 탭과 .md 다운로드가 보이는가
config.py	전원	.env 키만으로 모델이 열리는가
mocks/data.py	전원	샘플이 스키마와 맞는가

문서 버전은 저장소의 파일 구조와 같다. 파일명이나 Node 이름을 바꾸면 이 PDF를 `python tools/generate_guide_pdf.py` 로 다시 뽑고 팀에 공유한다.